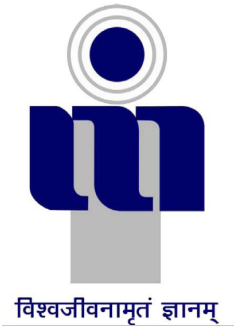
Automated Traffic Violation Detection System Using DNN

A project report submitted in partial fulfillment of the requirements for B.Tech. Project

B.Tech.

by

Maneesh Mondal (2019IMG-033)



ABV INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT
GWALIOR-474 015

2022

CANDIDATE DECLARATION

I hereby certify that the work, which is being presented in the report, entitled **Traffic Violation Detection System Using DNN**, in partial fulfillment of the requirement for the award of the Degree of **Integrated Post Graduate (IPG) Master of Business Administration** and submitted to the institution is an authentic record of our work carried out during the period *June 2022* to *September 2022* under the supervision of **Prof. Manisha Pattanaik**. I have also cited the reference to the text(s)/figure(s)/table(s) from where they have been taken.

Date:21/09/2022 Signature of the Candidate

This is to confirm that the candidate's above statement is true to the best of my knowledge.

Date:21/09/2022 Signatures of the Research Supervisors

ABSTRACT

In this work, a computer vision-based system for detecting traffic violations is presented. The technology picks up on infractions including going above the posted speed limit, parking without a permit, disobeying traffic signals.This traffic offence detection system consists of a pipeline of four different modules. These include a detection module for vehicles, a classification module for vehicles, a tracking module for vehicles, and a detection module for traffic violations. The vehicle detection module uses visual information, including live video feed, to identify vehicles on the road.Using a vehicle classification module, the cars are then categorised into several classifications after being spotted. To follow the car as it travels through the traffic, a vehicle tracking module is created. Finally,a traffic offence detection module has been designed that uses traffic patterns to evaluate and identify various traffic offences in real-time.The Deep Neural Network (DNN) module of OpenCV is used to construct the complete system.YOLOv4 model have been used to detect and classify the cars in the video frame.Vehicle detection accuracy measured in YOLOv4 is 86%.

Vehicles that are detected by YOLOv4 object detection models are tracked throughout the video frames. When a vehicle enters into the video frame for the first time, it is given a unique id and tracked between frames using DeepSORT [15]. Each vehicle's id and position in each frame are stored temporarily and analyzed for possible traffic offenses. We have implemented the DeepSORT algorithm in our system following the procedure of [16].

In this Traffic Offence Detection System three features are implemented using different various self made algorithm. The main three features are - Over speeding detection on road , detection of unauthorizedly parked car on road , detection of traffic signal violation.

Acknowledgement

This work has been a massive challenge, given that the field is still under research. It would not have been possible without the guidance and generous support of Prof. Manisha Pattanaik at Atal Bihari Vajpayee Indian Institute of Information Technology and Management, Gwalior. It is also because of resources provided by her that made this project possible. I thank Anil Rajput and Alok sir, for helping me with their vast knowledge and guiding me at every step; they helped a lot in successfully executing this task. At last, I would also like to thank all my colleagues who contributed something or other to the project.

[Maneesh Mondal]

TABLE OF CONTENTS

ABSTRACT ii

ACKNOWLEDGEMENT iii

LIST OF FIGURES v

ABBREVIATION viii

1 INTRODUCTION 1

1.1 Context 1

1.2 Motivation 3

1.3 Report Layout 4

2 LITERATURE REVIEW 5

2.1 Key Related Research 5

2.2 Research Gaps 7

2.3 Objectives 7

2.4 Summary 7

3 SYSTEM ARCHITECTURE AND METHODOLOGY 8

3.1 System Architecture 8

3.2 Methodology and Implementation 10

3.2.1 Vehicle Detection 10

3.2.2 Vehicle Tracking 11

3.2.3 Traffic Offence Detection 11

3.2.3.1 Speed Limit Violation Detection using Speed Estimation 11

3.2.3.2 Unauthorized Parking Detection On Road 16

3.2.3.3 Traffic Signal Violation Detection 19

3.3 Summary 22

4 RESULTS **23**

4.1 Activities Completed 23

4.2 Result Obtained 24

4.2.1 Vehicle Detection Using YOlOv4 24

4.2.2 Vehicle Tracking Using DeepSort Algorithm 25

4.2.3 Speed Limit Violation Detection Using Speed Estimation 26

4.2.4 Unauthorized Parking Detection 30

4.2.5 Traffic Signal Violation Detection 33

4.3 Summary 35

5 CONCLUSION AND FUTURE ASPECTS **36**

5.1 Conclusion 36

5.2 Future Scopes 37

REFERENCES **37**

LIST OF FIGURES

1.1 Comparison of the proposed YOLOv4 and other state-of-the-art object detectors. YOLOv4 runs twice faster than EfficientDet with comparable performance. Improves YOLOv3's AP and FPS by 10% and 12%, respectively[16] 2

3.1 Workflow of the Intelligent Traffic violation detection system 9
3.2 Types of violation detected by the traffic violation detection system . . . 10
3.3 Vehicle detection using YOLOv4,[17] 10
3.4 Results of tracking in MOTO16 challenge [14] 11

4.1 Vehicle Detected using YOLOv4,[15] 24
4.2 Vehicle Deteted using YOLOv4,[17] 24
4.3 Vehicle tracking using Deepsort,frame-1 25
4.4 Vehicle tracking using Deepsort,frame-2 25
4.5 Vehicle tracking using Deepsort,frame-3 25
4.6 Reference stripe on road 26
4.7 The car is tracked between to reference lines ,frame-1 26
4.8 The car is tracked between to reference lines ,frame-2 27
4.9 The car is tracked between to reference lines ,frame-3 27
4.10 The car is tracked between to reference lines ,frame-4 27
4.11 Individual folders storing the picture of individual vehicle 28
4.12 The particular folder of the vehicle that stored all the images of the vehicle that violates speed limit 28
4.13 Pictures of the car 29
4.14 Pictures of the bike 29
4.15 Excel sheet storing the vehicle details that violated the speed limit . . . 30
4.16 Region of interest 30
4.17 Timer has stated for the detected vehicle 31
4.18 Folder that stores the pictures of the illegally parked vehicle 31
4.19 The picture of the illegally parked vehicle 32
4.20 The excel sheet that stores the details of the illegally parked vehicle . . 32

4.21 Region of interest for traffic signal detection 33
4.22 Intersection of the bounding box of the vehicles and the region of the interest 33
4.23 Picture of bus in frame-1 34
4.24 picture of bus in frame-2 34
4.25 picture of car 34
4.26 Excel sheet that storing the details of the vehicle 35

ABBREVIATIONS

AI	Artificial Intelligence
CNN	Convolutional Neural Network
DNN	Deep Neural Network
CV	Computer Vision
GPU	Graphic Processing Unit
YOLOv4	You Only Look Once, Version 4

CHAPTER 1

INTRODUCTION

The number of automobiles now a days is rising quickly. This is one of the cause of congestion on the road. People may utilise a variety of modes of transportation, including automobiles, subways, and bicycles. However, despite all of these mobility options, cars continue to be the most popular owing to their comfort and practicality. In this approach, assuming constant population growth, the number of automobiles in big cities will also expand, but much more quickly than the transportation infrastructure; accordingly, traffic congestion will become a serious problem. It raises a variety of difficulties that are detrimental to the environment and society, including a rise in traffic accidents, economic growth, an increase in greenhouse gas emissions, time wasted, and health problems.By considering this matter this Machine learning-based traffic management systems have been suggested to identify and classify the cars, track them, and continuously analyse traffic characteristics for possible traffic violations.

A branch of artificial intelligence (AI), machine learning aids in both the identification and prediction of data as well as the forecasting of traffic. According to definitions, machine learning is a branch of AI that relies on taught input models to function. Based on a mathematical model, machine learning algorithms aid in the development of the intelligent traffic management system.

1.1 Context

In this project there is an application of YOLOv4 object detection model.You Only Look Once, or YOLO as it is sometimes abbreviated, is an object detection system that quickly and accurately detects a wide range of items inside a single enclosure. Additionally, it detects objects more quickly and precisely than earlier detection systems.Up to 9000 different types of visible and invisible elements may be estimated by it. YOLOv4 greatly outperforms the current approaches in terms of both "detection

performance" and "better speed". As per the Paper [1] describes it as a "speedily functioning" object detector that can be taught easily and utilised in production systems in relation to a publication that was published.The backbone of YOlOv4 is CSPDarknet53, Spatial Pyramid Pooling additional module, PANet path-aggregation,YOLOv3.It is an effective and reliable object identification model that enables users of 1080 Ti or 2080 Ti GPUs to train a very quick and precise object detector. YOLOv4 outperformed the quick and very accurate detectors in terms of "speed and accuracy," achieving an AP value of 43.5% (65.7% of AP50) over the Microsoft COCO dataset and gaining a real-time speed of over 65 FPS on the Tesla V100. For this reason I have used the YOLOv4 model for detecting the cars in this Project.To detect and classify the cars on the road, the video stream from the video camera is first examined using the YOLOv4 object identification model. Then, a variety of algorithms are used to find the traffic infringement.

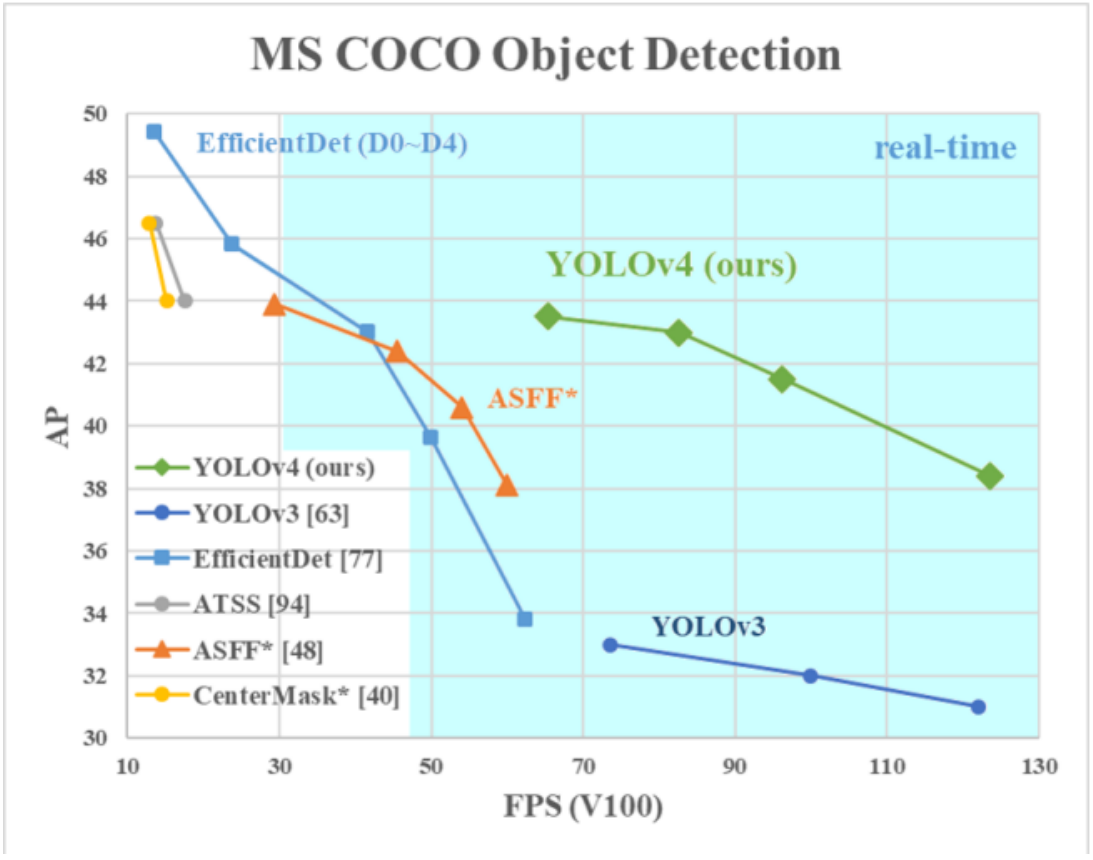


Figure 1.1: Comparison of the proposed YOLOv4 and other state-of-the-art object detectors. YOLOv4 runs twice faster than EfficientDet with comparable performance. Improves YOLOv3's AP and FPS by 10% and 12%, respectively[16]

1.2 Motivation

Modern society is quite concerned about traffic accidents. Every year, a lot of nations invest millions of dollars to lower the number of traffic accidents. Road users are protected from major injuries and fatalities by a variety of road safety techniques and regulations. However, every day a sizable number of accidents occur on the highways in many nations. Most of these collisions are caused by persons who are unwilling to abide by the laws of the road and by inadequate supervision. Research has revealed that for roads to be safe, changes must be made to the infrastructure, notably design standards that take into account the safety of all road users. According to [1], 112 nations have national design requirements for the control of speed. However, a lack of oversight and disregard for traffic regulations prevents infrastructure alone from making the roads safer. Currently, exciting and promising approaches to enhancing traffic monitoring and boosting road safety are provided by artificial intelligence methods in computer vision and machine learning.

1.3 Report Layout

Chapter 2 Literature Review describes the literature review which includes key related research with respect to the topic. The analysis of the existing work is also described which includes the contributions and the limitations of the referenced works. The objectives of the project have also been established in this part.

Chapter 3 Methodology discusses the methodology proposed by this project has been described in detail. This chapter also describes the brief derivation of the code of the project in detail.

Chapter 4 Results provides the result and the demo of this project for the better understanding.This section also includes the screenshots of the working application.

Chapter 5 Conclusion specifies the advantages,limitations and future scope of this project.

CHAPTER 2

LITERATURE REVIEW

This section is about the research papers and articles referred for any information and knowledge regarding the projects.

2.1 Key Related Research

A real-time estimation system for traffic parameters is necessary for traffic management. The research community is now working very hard to identify traffic metrics. Parallel process techniques are proposed by Ou et al [2] to detect the traffic offenses.Their approach used concurrent video feeds from many cameras to accomplish real-time traffic infraction identification in a monitoring stream. Three sorts of vehicles were intended to be detected by the system: speeding cars, cars on a blacklist, and cars with stolen licence plates. Using both actual and fake traffic data, the system was put into use and assessed.

Beymer et al [3] introduced a model that uses segmentation, classification, and tracking techniques to address traffic and lighting situations. The model focuses on monitoring and segmenting vehicles as well as computing traffic metrics from tracking data. To follow cars stuck in traffic, they developed a feature-based traffic tracking technique. The technology monitors vehicle sub-features, making partial occlusion less likely. A real-time version of the system has been built using a network of C40 DSP (Digital Signal Processor) chips connected to a host PC.

An strategy to employ an improved background-updating algorithm and feature-based tracking technique was put out by Wang et al. [4]. In their study, they employed feature-based tracking to follow the moving cars after using video-based traffic recognition with an enhanced background-updating algorithm.

A vehicle recognition and tracking volume statistics approach based on an enhanced single Gaussian model was put out by Zhu et al. [5]. Moving target identification, shadows suppression, and traffic volume count are the three components of the method. While shadow suppression is being done in the RGB feature space, a single Gaussian model is used to identify moving vehicles. The virtual lanes' traffic volume was determined.

On order to know the real-time measurements in urban roadways, Deng et al. [6] established a model that may be utilised for object segmentation, classification, and tracking techniques. To identify items that were fixed or moving slowly, the model used the background subtraction technique. Results from the experiments revealed a 20% decline in infrastructural capacity.

Wen-juan et al. [8] created a model that can identify and track the moving cars in a video and determine the traffic flow, the length of the line, the amount of time it takes to go through the line, the average speed, and other critical data. The model tracked objects using optical flow and online learning techniques. According to experimental findings, daylight detection accuracy was 98% and nighttime detection accuracy was 92%. Daytime average speed detection was 95%, whereas nighttime average speed detection was 90%.

Different Artificial Neural Network (ANN) models for real-time vehicle type detection were compared by Kim et al. [9]. For the best performance, they compared the processing speed and accuracy of the deep learning-based object identification models R-CNN (Region Based Convolutional Neural Network), Fast R-CNN, Faster R-CNN, YOLO, and SSD (Single Shot Detector). While SSD had greater FPS with less accuracy, faster R-CNN had better accuracy with less frames per second. The YOLO approach represented a compromise.

Based on the research on the modelling of neurological activity by Warren McCulloch and Walter Pitts, Zinchenko et al. [10] created an Artificial Neural Network (ANN) model. It adopts a similar strategy to Kim et al. [9] by using a quicker and more efficient model. The main goal of the proposed system is to provide better results in terms of shorter travel times, greater average vehicle velocity, and shorter wait periods for pedestrians and vehicles. Vehicle detection had a top and worst result of 94.65% and 88.71% respectively.

2.2 Research Gaps

After analysis of the above mentioned research we could find the following following gaps: Lists are easy to create:

1. The available research provides good approaches to tackle the problem of Traffic Violation detection but none of it talks about the implementation and deployment of the model so that it can be accessed by the end users.

2. There is no such model that detects efficiently detect Unauthorized parking on road.

2.3 Objectives

This project presents a computer vision-based system for traffic offense detection.The entire system is implemented using OpenCV Deep Neural Network (DNN) module. YOLOv4 have been used to detect vehicles on the roads with high accuracy.The main objectives of the projects are -

1. To detect the violation of Speed Limit On Road.

2. To detect the Unauthorizedly Parked Vehicle on busy roads.

3. To detect the violation of the traffic signal.

The main focus of the project is to make easy for the traffic police department to monitor the traffic.

2.4 Summary

This section is about the research papers and articles referred for any information and knowledge regarding the projects.Also the research gaps and the main objectives of the project is discussed in this section.

CHAPTER 3

SYSTEM ARCHITECTURE AND METHODOLOGY

The methodology is a relevant structure for research. We have multiple sections that-cover the Architecture, Working, and Set Up/Tools Used in the Project

3.1 System Architecture

The system uses a multi-stage processing approach to monitor and detect traffic violations from the video stream from traffic cameras.An object detection model(in this project YOlOv4 model is used for detection) first processes the video feed from the traffic camera to detect and classify the vehicles present in the video.Vehicles that are detected are monitored between frames,and all the available information from each frame of the video is stored temporarily and analyzed to detect any possible traffic offense.

The Project is mainly consists of two parts -

1. Vehicle detection
2. Vehicle tracking
3. Traffic offence detection

The Traffic offence detection is further divided into three parts-

1. Over speeding detection from speed estimation
2. Unauthorized parking detection
3. Traffic signal violation detection

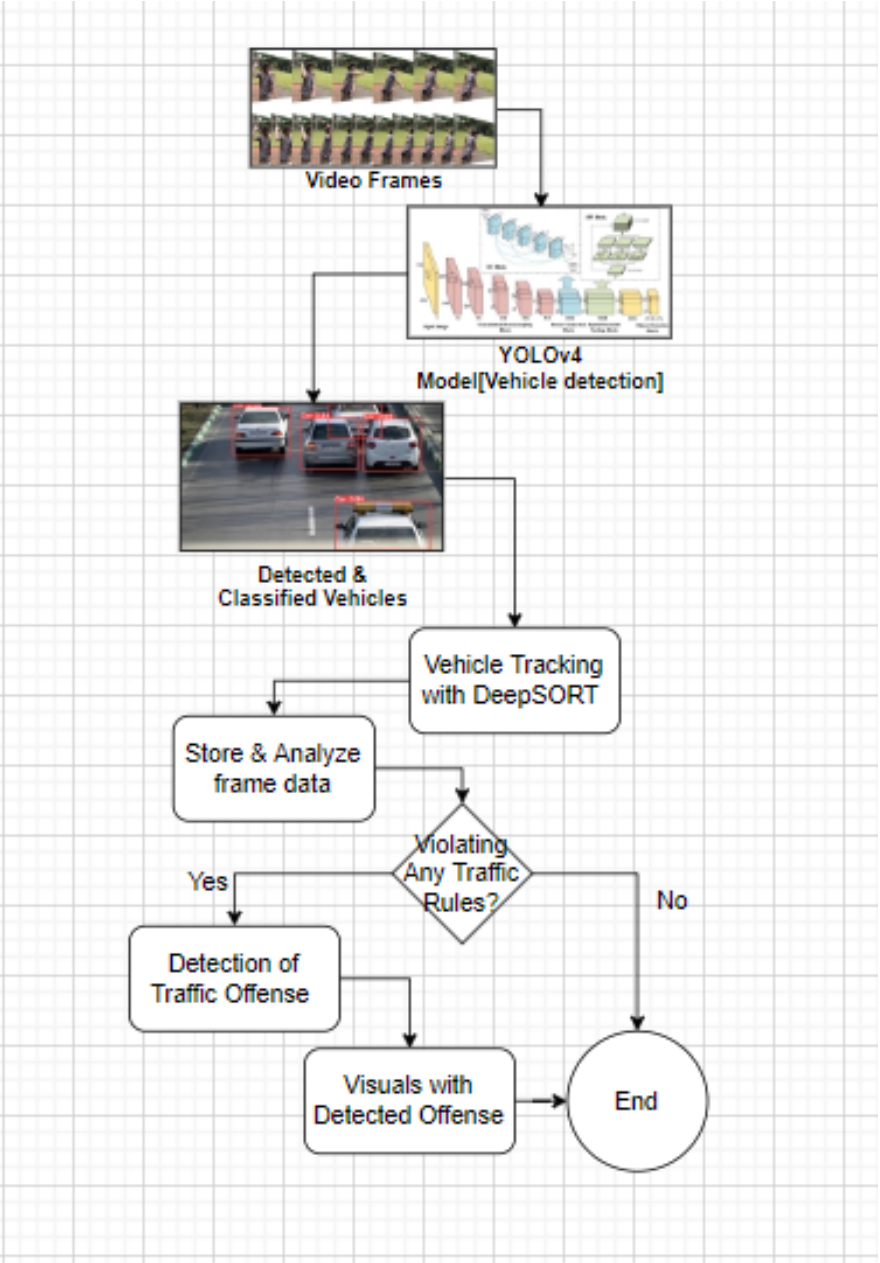


Figure 3.1: Workflow of the Intelligent Traffic violation detection system

Figure 3.1 shows a workflow of how the entire model would work starting from taking video frame as an input detecting and tracking of the vehicle and detection of the traffic violation and storing of the visuals of the detected offence.

Fig 3.2 shows the types of violation detected by the system including the speed limit violation detection, unauthorized parking detection,traffic signal violation

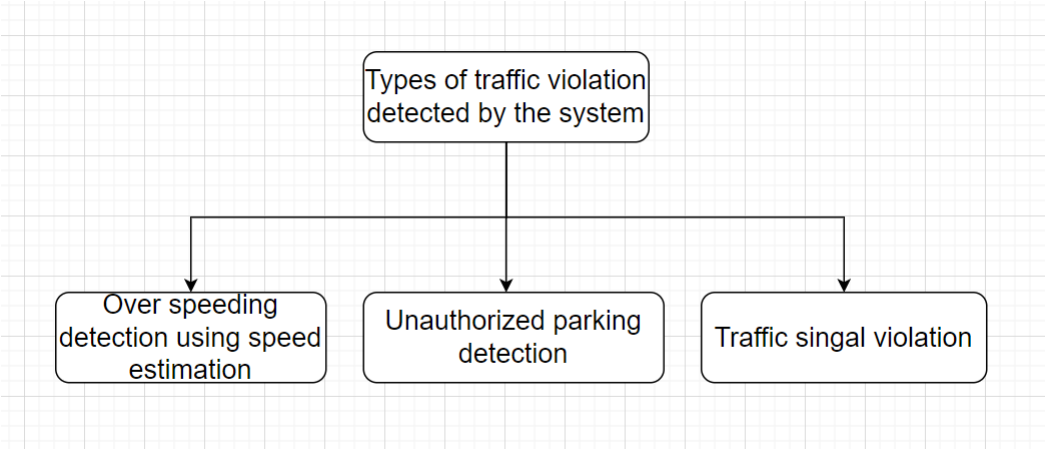


Figure 3.2: Types of violation detected by the traffic violation detection system

3.2 Methodology and Implementation

Methodology is a relevant structure for research. We have multiple sections that cover vehicle detection,vehicle tracking and traffic offence detection which included three parts-Over speeding detection using speed estimation,Unauthorized parking detection , Traffic signal detection.

3.2.1 Vehicle Detection

To detect the vehicle in the video, it is crucial to have an object detection model that can work effectively while giving adequate speed to monitor the traffic in real-time and accuracy to categorise the cars with respectable precision .With respectable speed and accuracy, YOLOv4 can identify vehicle and categorise them.

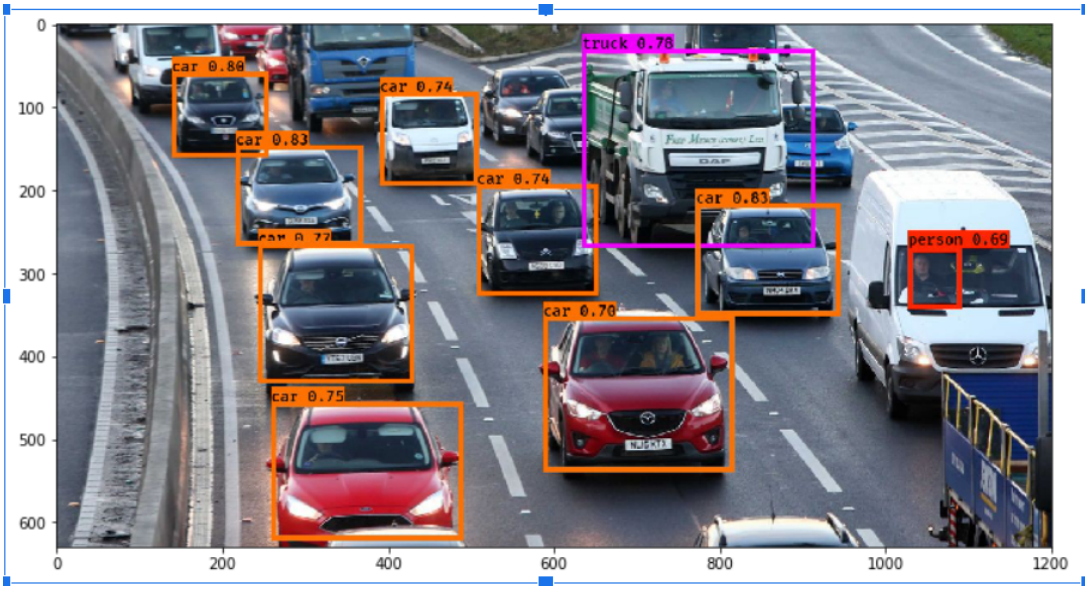


Figure 3.3: Vehicle detection using YOLOv4,[17]

Figure 3.3 shows the detected vehicle, named with unique id and model name by the YOLOv4 object detection model

3.2.2 Vehicle Tracking

Vehicles are tracked throughout the video frames using the YOLOv4 object detection models. A vehicle is assigned a unique id when it initially enters the video frame and is tracked between frames using DeepSORT [11]. The id and position of each vehicle in each frame are temporarily saved and analyzed for possible traffic violations.We implemented the DeepSORT algorithm in my project following the procedure of [12], [13].

		MOTA ↑	MOTP ↑	MT ↑	ML ↓	ID ↓	FM ↓	FP ↓	FN ↓	Runtime ↑
KDNT [16]*	BATCH	68.2	79.4	41.0%	19.0%	933	1093	11479	45605	0.7 Hz
LMP_p [17]*	BATCH	**71.0**	**80.2**	**46.9%**	21.9%	434	**587**	7880	**44564**	0.5 Hz
MCMOT_HDM [18]	BATCH	62.4	78.3	31.5%	24.2%	1394	1318	9855	57257	35 Hz
NOMTwSDP16 [19]	BATCH	62.2	79.6	32.5%	31.1%	**406**	642	**5119**	63352	3 Hz
EAMTT [20]	**ONLINE**	52.5	78.8	19.0%	34.9%	910	**1321**	**4407**	81223	12 Hz
POI [16]*	**ONLINE**	**66.1**	79.5	**34.0%**	20.8%	805	3093	5061	**55914**	10 Hz
SORT [12]*	**ONLINE**	59.8	**79.6**	25.4%	22.7%	1423	1835	8698	63245	**60 Hz**
Deep SORT (Ours)*	**ONLINE**	61.4	79.1	32.8%	**18.2%**	**781**	2008	12852	56668	40 Hz

Figure 3.4: Results of tracking in MOTO16 challenge [14]

The comparison to other published methods with non-standard detection. The full table of results can be found on the challenge website.]. This benchmark evaluates tracking performance on seven challenging test sequences.

3.2.3 Traffic Offence Detection

In the section we basically discussed briefly about how the various kinds of traffic offence detection can be implemented.

3.2.3.1 Speed Limit Violation Detection using Speed Estimation

After the vehicle tracking system is implemented using the Deep-Sort Algorithm and the accompanying object detection has been occurred using pre-trained Yolov4 weights, we are able to differentiate between each vehicle.The main logic to implement this is based on simple physics formula. That is:

$$v = d/t \tag{3.1}$$

where:
v = Velocity of the vehicle
d = Distance covered by the car
t = Time taken to cover the distance

Algorithm 1 Over Speeding Detection

```
1:  function detectSpeedViolation(id, bbox, gap, st1, st2, speedLimit)
2:  vehicle_times =[]
3:  violations_detected = []
4:      if    bbox ∩ st1 ≠ ∅ then
5:          if   id  not  in  vehicle_times then start_time = now()
6:              vehicle_times[id] = -1 * start_time
7:              end if
8:  end if
9:      if    bbox ∩ st2 ≠ ∅ then
10:         if   id  in  vehicle_times then
11:             if   id  not  in  violation_detected then
12:                 finish_time = now()
13:                 speed=gap/(vehicle_times[id]+finish_time)
14:                 if speed > speedLimit then
15:                     violation_detected[id]=[speed, now()]
16:                     end if
17:                 end if
18:             end if
19:         end if
20:     =0
```

First, two reference points are located —or, to be more precise, two reference lines. In addition,the separation between the two lines are known. Since our camera input contains no depth information at all, this distance must be precisely understood.

For appending the pixel coordinates of the two reference points two empty arrays have been constructed assuming the name is poly_co_start, poly_co_finished. Here , a stripe of pixel coordinates is used rather than a single pixel line as a reference line since sometimes an intersection would not happen if the reference lines are too thin. As a result, there is no junction as the car crosses the reference line.

Initially two blank masks are made assuming the name is pts_start, pts_end having the size corresponding to the video resolution. (Which also matched the size of the frame) and provided the coordinate points for my two rectangle stripes as an array in those blank musks. Then, I marked all of the pixel coordinates in those blank musks with the value 255 using the fillpoly function in cv2.

4 variables are declared to get all of the pixel coordinates at the binary image having the pixel value of 255. As of right now, we know the coordinate values of the pixels along the reference line (stripe) that corresponded to the correct spot in the original

video clip. Therefore , all of the coordinate values are iterated for the pixels in the two reference stripes to create an array for the two reference stripes that we can use to determine where the intersection of a mini-bounding box of a vehicle will be occred.

Then a for-loop is taken in which just one car was taken into account. Although all of the cars are being monitored at once, for the sake of simplicity, we may assume that there is only one vehicle on the road and that all of the functions we create only apply to it. Naturally, all vehicles simultaneously benefit from this feature, but you don't need to worry about doing so. Not the code itself, but conceptual complexity is reduced as a result.

Consider that one car is now crossing the first reference line. Then a variable is taken that contains the bounding box coordinates of the vehicle in the xmin, ymin, xmax, ymax format. Also there are two variables respectively , containing the vehicle type and the tracking id.The bounding box in YOLO object detection is considerably bigger in the vehicles that are near to the camera than in the vehicles that are distant from the camera's field of view. This causes extra acceleration, which causes an issue when determining the speed. Since the bounding box in YOLO serves as a representation of the actual vehicle, I've observed situations where the increased bounding box (caused by getting close to the camera) intersects the second reference line much earlier than the actual vehicle would have expected, resulting in a much higher speed than intended. So I computed the centre coordinates of that bounding box.As of right now, the bounding box we have simply represents the vehicle and not even the surrounding open area. This is done by building a mini-bounding box using the central x and y coordinates that always remains within the centre of the vehicle and doesn't depict any empty space around it or increase larger each time the vehicle gets near to the camera.

Then an array is created assuming the name b_box_co and added only that mini, constant vehicle bounding box coordinates to it. 10-pixel points are taken from the top, bottom, left, and right to the centre coordinates points that were previously specified in order to create the mini_bounding box.I took into consideration each coordinate point on the mini bounding box because I needed to generate an array of pixel coordinates to show where the reference line coordinates and the newly produced mini-bounding box coordinates intersect.

The vehicle name, its tracking ID, and its timestamp from the Python "datetime" package are added to an array in order to record when the car crossed the first reference line. This array, which I called the cache_matrix would have the following format attached

to it when the intersection occurred at the first line.

The circumstances have been determined for when a car initially crosses the first reference line. According to the programming, if the poly co start and the b box co have any comparable coordinate points, and if there is at least one of those points in common, add the t_start(it the time stamp that has taken from local time of computer at the time of intersecting), vehicle type, and tracking id to the cache matrix only if it hasn't already been attached to the cache matrix.

There are many frames that show the intersection of the bounding box and referenced stripe, which results in appending a number of incorrect entries to the cache matrix. Therefore, we need to make sure that the append only happens once, and that would be when the vehicle first crosses the reference stripe.

Imagine that the car maintains driving toward the second reference strip after the first intersection and the entry logging. The following requirements and actions had to be completed before the car could cross at the junction just after the second one.

In order to identify the individuals who went above the set speed limit,an array have been added assuming the name viol_detect.

If the pixel coordinate values for poly_co_finish and b_box_co have at least one pixel in common, the condition is true that the vehicle intersects the 2nd stripe.Then I check the vehicle type and vehicle tracking id in the cache matrix to see whether that vehicle has completed its first intersection. To prevent completing more than one intersection, look for any prior append entries in the violation matrix pertaining to the same intersection.

If the result of this if-condition is True, the time is instantly checked and added to a variable called t_finish. The t_start value must then be extracted from the cache matrix since, given that the if condition was satisfied, we are certain that such an item exists in the cache matrix array. I initially established a variable named index that serves as a milestone in order to simplify the extraction. We can get the location of the beginning string for the vehicle type and the tracking id of the cache matrix by reading the cache matrix inside this as a string and using the find function. The value is placed at certain and fixed distances from the text denoting the vehicle class since the time we recorded is in one of the common time formats ('%y-%m-%d%H:%M:%S'). In this instance, we may locate the t_start in its usual format by using the index value that we already determined between 28 and 11 characters to the left in the variable t_start_cm. But because t_start_cm is a string once again, we must convert it to a timestamp using "datetime"

in order to figure out how much time has passed. "strptime" method and added it to a variable assuming the name date_time_obj field.

Since we can determine the time difference, we can now calculate the speed.

Since we require the time difference in seconds, a very simple speed function is written and sent the time as an attribute, using the total seconds() method as the input of the function. In this instance, 10m is used as the distance between two reference stripes.

Now that the vehicle velocity has been properly retrieved, we can use an if condition to find those who have broken the speed limit. All the cars are picked with a speed restriction above 10 for demonstrative reasons.

Our findings are then recorded on a spreadsheet so that anybody can subsequently see when the vehicle was initially spotted, its speed at the time of the infringement, its class, and its kind. To prevent the replacement of the log due to more than one vehicle being taken into consideration at once, I have put these three entries in three columns, one row, and made sure to raise the row number by one. The declaration for the spreadsheet is as follows.

Now that the information of the violating vehicle, including the start_time, rounded velocity, class name, and tracking id, has been added,by which we can check whether the same vehicle has not been detected more than once.

So, we need to take a picture of this car as soon as it commits the offense in order to input it into the system for recognising licence plates and to be reviewed manually as evidence.

To ensure that the cropping has caught all the data necessary for the licence plate identification and recognition, I raised the picture's size (xmax and ymax) coordinates by 100 pixels and cropped the image using the original bounding box coordinates (not the mini-bounding box coordinates). I then named the cropped picture with its class name and tracking id and stored it in an output folder.
(Example: car3.jpg.)

The system will then take frame-by-frame snapshots of the infringing cars and store them to a special folder with the vehicle's class type and tracking ID as its name. Therefore, the picture will be cropped first if a vehicle is found and added to the viol detect

array. Then it looks to see whether a directory with the name and kind of car already exists. In order to prevent replacements from new detection, if there isn't already a directory with that name, the application first creates one before saving the picture with that name as the frame number(Since a single frame cannot contain more than one instance of the same vehicle.).

This is how the whole process of speed limit violation detection or the over speeding detection has been implemented.

3.2.3.2 Unauthorized Parking Detection On Road

Algorithm 2 Over Speeding Detection

```
1:  procedure MOBILITY(previous_box_coords, current_box_coord)
2:
3:      if (current_box_coord − previous_box_coords) < 4 then
4:          Return  True
5:      end if
6:  end procedure
7:  procedure PARKINGVIOLATION(tracking_id, bbox_coords, stripe_cords, time)
8:  parking_times=[]
9:  frame_matrix=[]
10: violations_detected = []
11:     for Every frame
12:     if  bbox ∩ stripe_cords ≠ ∅ then
13:         if  tracking_id not in frame_matrix then
14:             frame_matrix[tracking_id] = frame_num,bbox_cords
15:             previous_box_coords=frame_matrix[frame_num-1]
16:             if mobility(previous_box_coords,bbox_coords)returns true then
17:                 t_start=now();
18:                 parking_times[tracking_id]=t_start,classname
19:                 if tracking_id is not in violations_detected[] then
20:                     t_spending=now()-t_start
21:                     if t_spending > time then
22:                         violations_detected[now(),classname]
23:                     end if
24:                     end if
25:                 end if
26:             end if
27:         end if
28:     end procedure=0
```

After the object detection has occurred using pre-trained Yolov4 weights, we are able to differentiate between each vehicle.Now the main implementation part is described below.

First in the video frame we have to select a pixel coordinate of a parking space where any car is not supposed to be parked.Therefore, a zero array is established that is the same size as the video resolution after initially introducing a blank array named parking_co. Then choose the polygon's coordinates according to the shape you chose as the area of interest (ROI).Then an array named pts_array is introduced where the edge coordinate of the polygon has been stored.But this is insufficient. To identify when the vehicle (bounding box) coordinates cross them, I need the pixel coordinates that cover the inside of this polygon. For that, I used the "fillPoly" function of "OpenCV" and supplied the coordinates from the pts array as one of its properties.

The blanked mask was then filled with the pixel values of the region of interest after I retrieved the x and y coordinates points.

Now all of the pixel coordinate locationsare stored in the region of interest or the area where parking is not permitted for cars. Next, let's take the vehicle's own bounding box coordinates. However, this raises a new issue. The frame rate decreases significantly if the camera is too near to the region of interest because when a vehicle approaches that area, its bounding box takes up a lot of coordinate points, and when many cars are present, this procedure must be repeated for each frame of the video,as a result the frame rate drops drastically. So I have made the case that if a vehicle or bounding box intersects this ROI, it very certainly also does so with the bounding box's bottom line. Therefore, similar to the lane line violation, I have simply used the location of the bottom line of the bounding box instead of all the vehicle's bounding box coordinates, which not only supports my arguments but also increases the frame rate (performance).

Now as told earlier the bottom line coordinates have been extracted and for a better depiction of the vehicle, I have cut the length of the line by subtracting 50 pixel coordinates from each side of the line. As a result, the line never infringes on any surrounding area and always remains in the region of the car.

Now consider the first time a car would pass through the area. When it performs the intersection of ROI during a particular frame and instantly add the frame number of that incident, the vehicle type (class name), the vehicle tracking id (str(t)), and the bounding box coordinates of that vehicle at that frame to a matrix name fame_matrix.

Then it compares the prior frame to see whether the same kind of intersection for the same vehicle happened. If this intersection occurred for the first time, the program moves on to the next frame without additional interaction since the condition is not met. But suppose that a value is truly obtained for a variable named check index in the

second frame after the intersection of the vehicle with the region of interest.

The boolean value of (check_index+1) returns a positive number. The reasoning for the addition of 1 is because it is feasible that a particular vehicle's logging information begins at the very beginning of the array, causing its placement value in the array to be 0. In such a case, even if the condition is True, bool(0) renders it false. When a +1 is added to the -1 that check_index provides when there isn't a matching log entry, it produces the required False response.

As you can see, when that condition is met, they got the bounding box data from the previous frame log and stored them in a new variable named Previous_bbox_co_str.

Now that we are aware of the vehicle's bounding box coordinates at both the present and the prior intersections. Since we are aware that the car has been in the ROI for a long period of time, we must evaluate if it is fixed (immobile) or moving. To make sure about that have been defined the immobility function.

As a result, both the current bounding box coordinates and the previous bbox co str have been used as attributes to the immobility function. The difference of the corresponding xmin, ymin, xmax, and ymax values have been taken. If the absolute difference is less than 4, I have outputted True, indicating that the vehicle is immobile, and False, indicating the countermeasure.

Another issue is that YOLO often provides shifting bounding box coordinates, even when a vehicle or other object is completely stopped. I have chosen a bigger value for the total variable in this case to prevent and make the procedure robust. The software becomes more resistant to the random fluctuations of the bounding boxes the higher the value.

Therefore, if the function returns that the vehicle is stationary (immobile), Immediately it is checked to see whether there had been any earlier recorded entries in the cache_matrix; if not, I appended the current time (t_start), the vehicle type , and the tracking id to the cache_matrix.

Now it is determined if the vehicle is in the cache matrix for the subsequent condition. (Now it is.) Additionally, make sure the same vehicle is not included in the viol_matrix. (Because I've applied the infraction matrix to recorded cars that were stationary and went over the time restriction.) If not,got instantly get the t_start value for that individual car and use it to verify whether the time has changed using the current

time.

Vehicle initial intersection time (t start cm), vehicle type (class name), and vehicle tracking ID (str(t)) were all recorded in a spreadsheet if the time difference (t spending) exceeded the required time limit. In this instance, I've set the timer for 10 seconds to serve as an example.

The car has now committed a violation, thus we must add a log record for this particular vehicle to the viol matrix array. And by doing so, more repetition and faulty log entries about a vehicle being there even after a violation has happened are avoided. Once that car has been infringed, you may take a picture of it and store it with its unique vehicle type and tracking ID as its name.
Now, this is essentially how the parking infraction detection method that I have implemented .

3.2.3.3 Traffic Signal Violation Detection

Algorithm 3 Traffic Signal Violaiton Detection

```
1: procedure TRAFFICSIGNALVIOLATION(id, bbox, stripe_cords)
2: violations_detected = []
3:     if   bbox ∩ st1 ≠ ∅ then
4:         if   id not in violations_detected then start_time = now()
5:             violations_detected[id] = start_time
6:             end if
7: end if
8:     =0
```

After the vehicle tracking system is implemented using the Deep-Sort Algorithm and the accompanying object detection has occurred using pre-trained Yolov4 weights, we are able to differentiate between each vehicle. First, we must locate one reference point—or, to be more precise, one reference line.

For appending the pixel coordinates of the reference points an empty array have been constructed assuming the name is poly_co_start. Here,a stripe of pixel coordinates have been used rather than a single pixel line as a reference line since sometimes an intersection would not happen if the reference lines are too thin. As a result, there is no junction as the car crosses the reference line.Here the reference line occures when the signal is red.That basically symbolises the red singal.

Initially made a blank mask assuming the name is pts_start having the size corresponding to the video resolution. (Which also matched the size of the frame) and provided the coordinate points for the rectangle stripe as an array in that blank musk. Then, I marked all of the pixel coordinates in that blank musk with the value 255 using the "fillpoly" function in "cv2".

Then 2 variables are added to get all of the pixel coordinates at the binary image having the pixel value of 255. As of right now, we know the coordinate values of the pixels along the reference line (stripe) that corresponded to the correct spot in the original video clip. Therefore, I iterated all of the coordinate values for the pixels in the reference stripe to create an array for the reference stripe that we can use to determine whether the intersection of a mini-bounding box of a vehicle will be occred or not.

A for-loop is taken in which just one car was taken into account. Although all of the cars are being monitored at once, for the sake of simplicity, we may assume that there is only one vehicle on the road and that all of the functions we create only apply to it. Naturally, all vehicles simultaneously benefit from this feature, but you don't need to worry about doing so. Not the code itself, but conceptual complexity is reduced as a result.

Consider that one car is now crossing the first reference line. Then a variable is taken that contains the bounding box coordinates of the vehicle in the xmin, ymin, xmax, ymax format. Also there are two variables respectively , containing the vehicle type and the tracking id.The bounding box in YOLO object detection is considerably bigger in the vehicles that are near to the camera than in the vehicles that are distant from the camera's field of view. This causes extra acceleration, which causes an issue when determining the speed. Since the bounding box in YOLO serves as a representation of the actual vehicle, I've observed situations where the increased bounding box (caused by getting close to the camera) intersects the second reference line much earlier than the actual vehicle would have expected, resulting in a much higher speed than intended. So I computed the centre coordinates of that bounding box.As of right now, the bounding box we have simply represents the vehicle and not even the surrounding open area. This is done by building a mini-bounding box using the central x and y coordinates that always remains within the centre of the vehicle and doesn't depict any empty space around it or increase larger each time the vehicle gets near to the camera.

Then an array is created assuming the name b_box_co and added only that mini, constant vehicle bounding box coordinates to it. 10-pixel points have been taken from

the top, bottom, left, and right to the centre coordinates points that were previously specified in order to create the mini_bounding box.I took into consideration each coordinate point on the mini bounding box because it needed to generate an array of pixel coordinates to show where the reference line coordinates and the newly produced mini-bounding box coordinates intersect.

The vehicle name, its tracking ID, and its timestamp from the Python "datetime" package are added to an array in order to record when the car crossed the first reference line. This array, which I called the "cache matrix," would have the following format attached to it when the intersection occurred at the first line.

The circumstances have been determined for when a car initially crosses the first reference line. According to the programming, if the poly co start and the b box co have any comparable coordinate points, and if there is at least one of those points in common, add the t_start(it the time stamp that has taken from local time of computer at the time of intersecting), vehicle type, and tracking id to the cache matrix only if it hasn't already been attached to the cache matrix.

There are many frames that show the intersection of the bounding box and referenced stripe, which results in appending a number of incorrect entries to the cache matrix. Therefore, we need to make sure that the append only happens once, and that would be when the vehicle first crosses the reference stripe.

After the violation is detected we simply add the timestamp (that can be found by extracting the value of t_start) and the car unique id and model no in the excel sheet.

Now it need to take a picture of this car as soon as it commits the offense in order to input it into the system for recognising licence plates and to be reviewed manually as evidence.

To ensure that the cropping has caught all the data necessary for the licence plate identification and recognition and raised the picture's size (xmax and ymax) coordinates by 100 pixels and cropped the image using the original bounding box coordinates (not the mini-bounding box coordinates). Then named the cropped picture with its class name and tracking id and stored it in an output folder.
(Example: car3.jpg.)

The system will then take frame-by-frame snapshots of the infringing cars and store them to a special folder with the vehicle's class type and tracking ID as its name. There-

fore, the picture will be cropped first if a vehicle is found and added to the viol detect array. Then it looks to see whether a directory with the name and kind of car already exists. In order to prevent replacements from new detection, if there isn't already a directory with that name, the application first creates one before saving the picture with that name as the frame number(Since a single frame cannot contain more than one instance of the same vehicle.).
This is how the whole process of Traffic signal violation detdction has been implemented.

3.3 Summary

In this chapter mainly the methodology and the algorithm of the three objectives of the projects are described briefly.
Firstly the the implementation of vehicle detection model is described using various pictures.
Then the working process of the vehicle tracking algorithm (Deepsort Algorithm) is described .
Then the working process of traffic violation detection system is described in three sub parts - Over speeding Detection , Detection of unauthorizedly parked car, Traffic signal violation detection.

CHAPTER 4

RESULTS

We have now successfully executed the concept of the project and now the Intermediate Results of the Project and what activities are to be done are displayed over here.

4.1 Activities Completed

1. Filtered recent research papers on object detection,traffic violation detection,studied on increasing number traffic violation and its negetive effetcs on the human society.
2. Selected some research paper and did the literature review.
3. From the literature review, found out the common problems faced by each model and generated the problem statement.
4. Used the YOLOv4 Object detection model to detect the vehicle in the video frame.
5. Used the Deepsort Algorithm to track the vehicle through the video frame.
6. Designed the flowchart of working.
7. Implemented the various kinds of detection process of traffic offence.
8. Storing the details of the vehicle that violated the traffic rules.

4.2 Result Obtained

This section elaborates the all the outputs achieved till now after implementing all of the Traffic Violation detection algorithms.

4.2.1 Vehicle Detection Using YOlOv4

In the this section the screenshots from Vehicle Detection are presented.



Figure 4.1: Vehicle Detected using YOLOv4,[15]

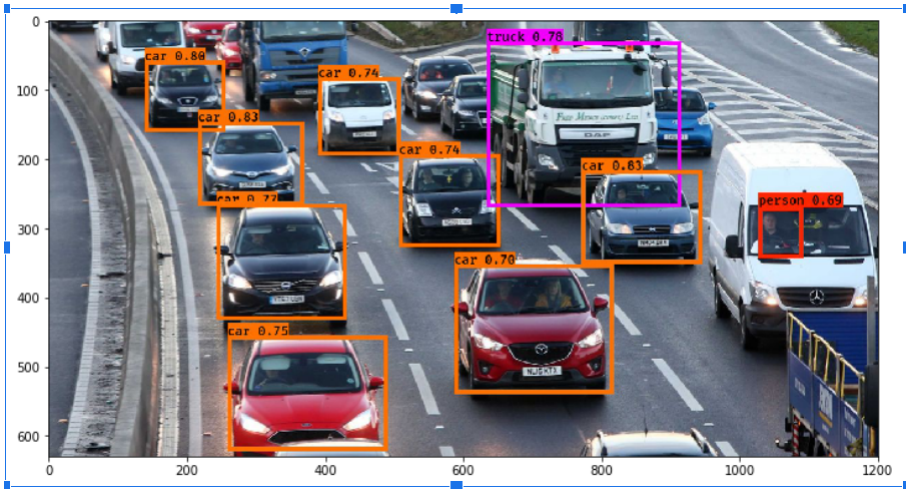


Figure 4.2: Vehicle Deteted using YOLOv4,[17]

We can see in both images (4.1,4.2) shows the cars are detected with unique id and the class name.

4.2.2 Vehicle Tracking Using DeepSort Algorithm



Figure 4.3: Vehicle tracking using Deepsort,frame-1



Figure 4.4: Vehicle tracking using Deepsort,frame-2



Figure 4.5: Vehicle tracking using Deepsort,frame-3

In the figures (Figure 4.3, Figure 4.4,Figure 4.5) we can see here the detected vehicles are tracked in several consecutive frames using Deepsort algorithm.

4.2.3 Speed Limit Violation Detection Using Speed Estimation

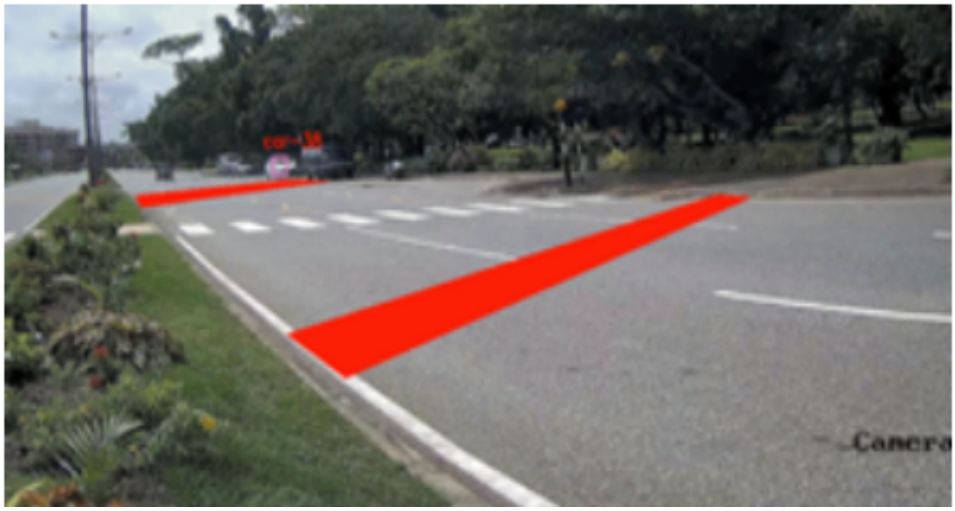


Figure 4.6: Reference stripe on road

As discussed in the methodology these are the two reference stripe(Figure 4.6). The distance between two stripes are 10 meter.This two stripes are used to measure the speed of the vehicle.

Now if any vehicle violates the speed limit The system will then take frame-by-frame snapshots of the infringing cars and store them to a special folder with the vehicle's class type and tracking ID as its name.

Also the system records the vehicle type, vehicle tracking ID and the vehicle speed in the spreadsheet too for the license plate recognition.



Figure 4.7: The car is tracked between to reference lines ,frame-1



Figure 4.8: The car is tracked between to reference lines ,frame-2



Figure 4.9: The car is tracked between to reference lines ,frame-3



Figure 4.10: The car is tracked between to reference lines ,frame-4

All the figures (Figure4.7,Figure 4.8 ,Figure 4.9 ,Figure 4.10) show how the movement of the vehicle is tracked between two reference stripes in various frames.

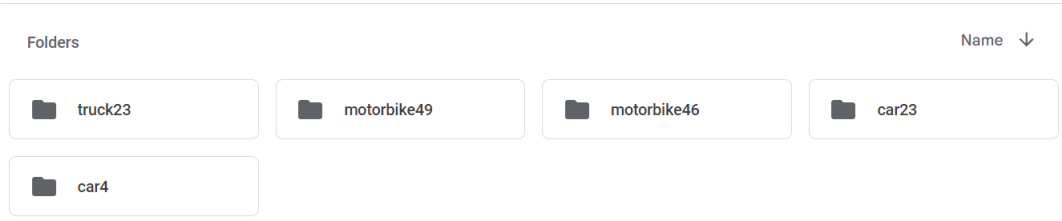


Figure 4.11: Individual folders storing the picture of individual vehicle

These are those individual folders named as the vehicle tracking id and class where the pictures of the vehicles that violated the speed limit from different frames are stored . This picture(figure 4.12) shows that all the pictures of the particular car from individ-

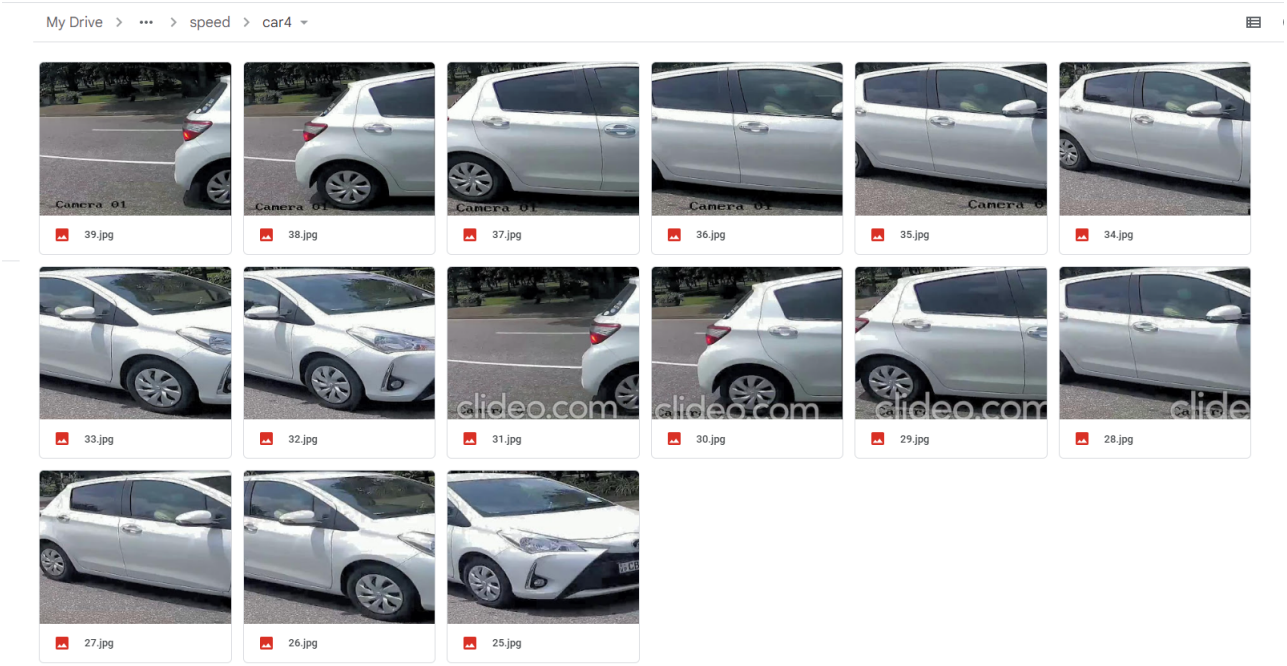


Figure 4.12: The particular folder of the vehicle that stored all the images of the vehicle that violates speed limit

ual frame that violate the speed limit are stored in the folder which name is same as the tracking id and class of the particular car.



Figure 4.13: Pictures of the car



Figure 4.14: Pictures of the bike

This is how (Figure 4.13,Figure 4.14) the the systems stores many the pictures of the vehicle that violated the speed limit.With the help of those picture the authority easily can find the details of the vehicle.

Time of violation	Vehicle speed	Tracking id
22-09-16 13:52:06	137.09	truck63
22-09-16 13:49:01	12.36	truck25
22-09-16 13:51:08	53.23	motorbike48

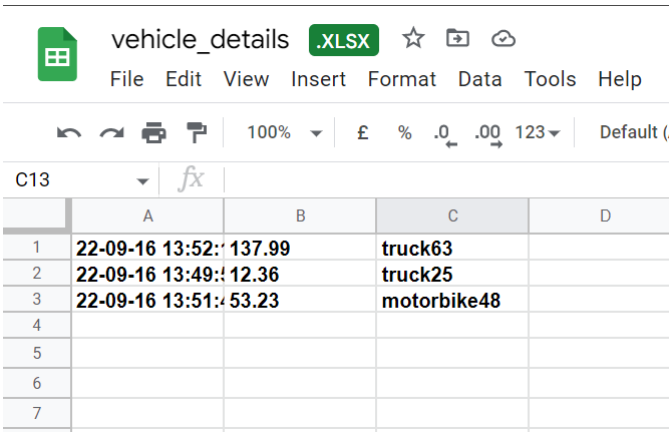


Figure 4.15: Excel sheet storing the vehicle details that violated the speed limit

Figure 4.15 shows the excel sheet where the details of the vehicle that violated the speed limit.In the A column the time when the vehicle touched the 1st stripe is stored. In B column the speed of the vehicle is stored, basically for demonstration purpose the threshold speed is set to 10.The column the c the vehicle tracking id and class name is stored.

4.2.4 Unauthorized Parking Detection

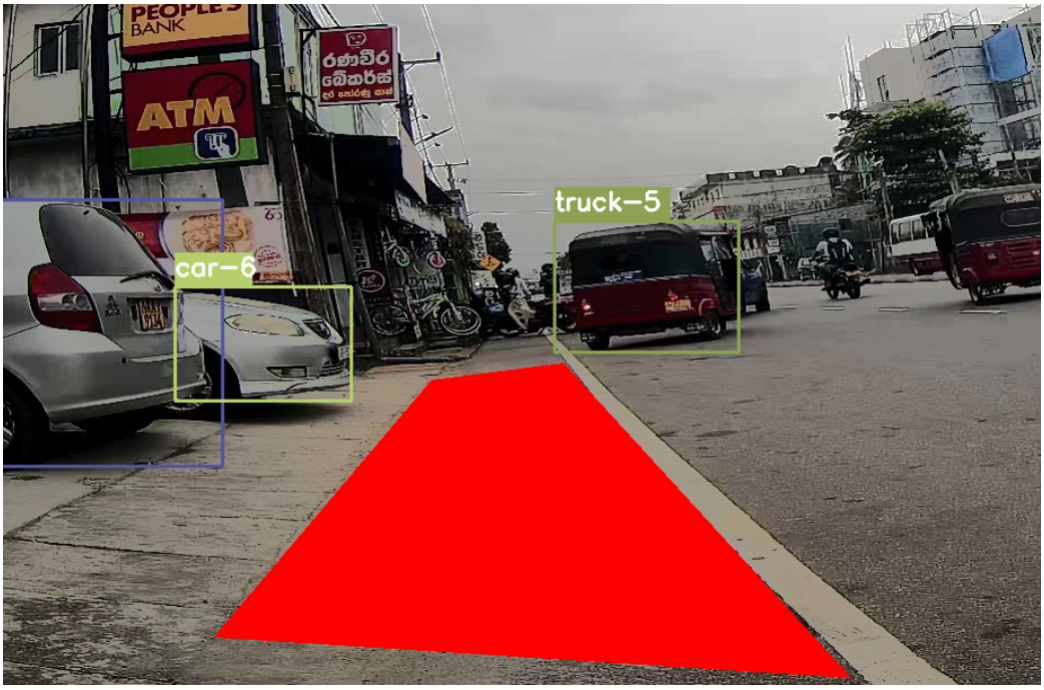


Figure 4.16: Region of interest

As discussed in the methodology firstly the region of the interest is created where any vehicle are not allowed to be parked.Figure 4.16 shows that region of interest.

In figure 4.17 we can see when the bottom line of the bounding box intersects the



Figure 4.17: Timer has stated for the detected vehicle

region of interest the car is given a tracking id and timer is started for the detected car. After that if the time limit exceeds the threshold time(which is set here 5 seconds for demonstration purpose) the car is considered as illegally parked.

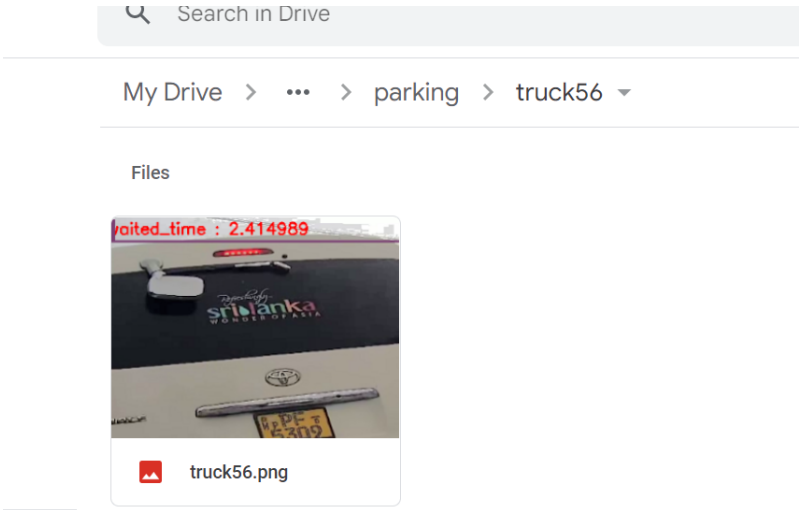


Figure 4.18: Folder that stores the pictures of the illegally parked vehicle



Figure 4.19: The picture of the illegally parked vehicle

Here we can see (figure 4.19) the clear picture of the vehicle that is illegally parked is stored in the database.

Time of violation	Tracking id
22-09-16 13:51:08	truck56

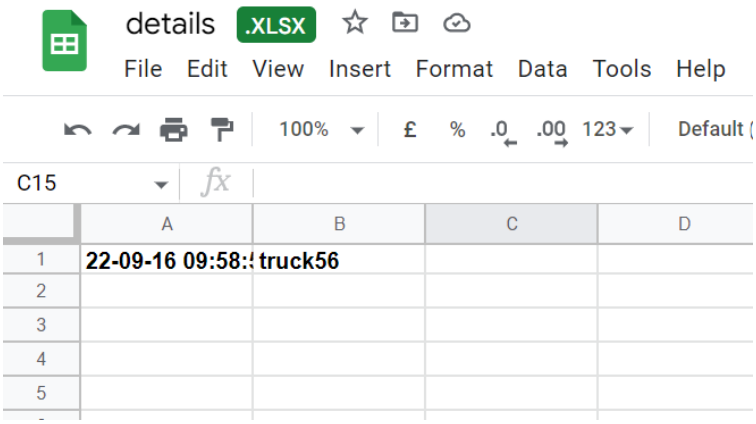


Figure 4.20: The excel sheet that stores the details of the illegally parked vehicle

In the figure 4.20 we ca see the details of the car is recorded in the excel sheet. The A column stores the date and time of parking and the B column stores the unique id and the class name of the detected vehicle.

4.2.5 Traffic Signal Violation Detection

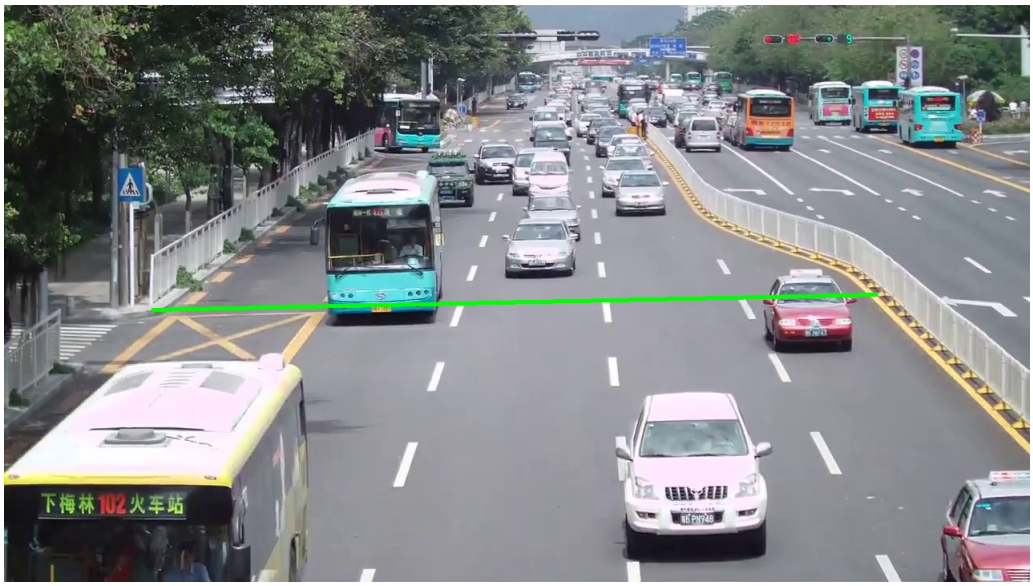


Figure 4.21: Region of interest for traffic signal detection

In the figure 4.21 shows the region of the interest on the road which is considered as a red signal. If any bounding box of the vehicle intersect the region of interest then it is considered as violation of traffic signal. Here we can see in figure 4.22 few bounding

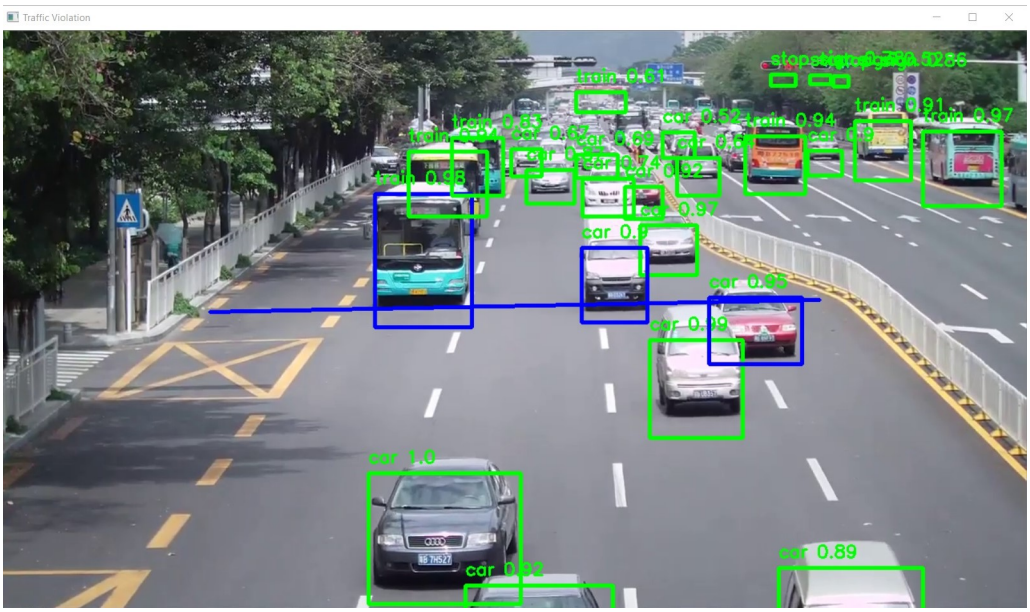


Figure 4.22: Intersection of the bounding box of the vehicles and the region of the interest

boxes intersected the region of interest so it is considered as the violation of the traffic signal.Also the colour of the bounding boxes that intersected the region of interest are changed to blue.



Figure 4.23: Picture of bus in frame-1



Figure 4.24: picture of bus in frame-2



Figure 4.25: picture of car

These are the Pictures(Figure 4.23, Figure 4.24,Figure 4.25) of the vehicles that are detected as violating the traffic signal.All the pictures are stored in the database.

Time of violation	Vehicle Tracking id
22-09-17 13:54:01	bus3
22-09-17 13:54:08	car12

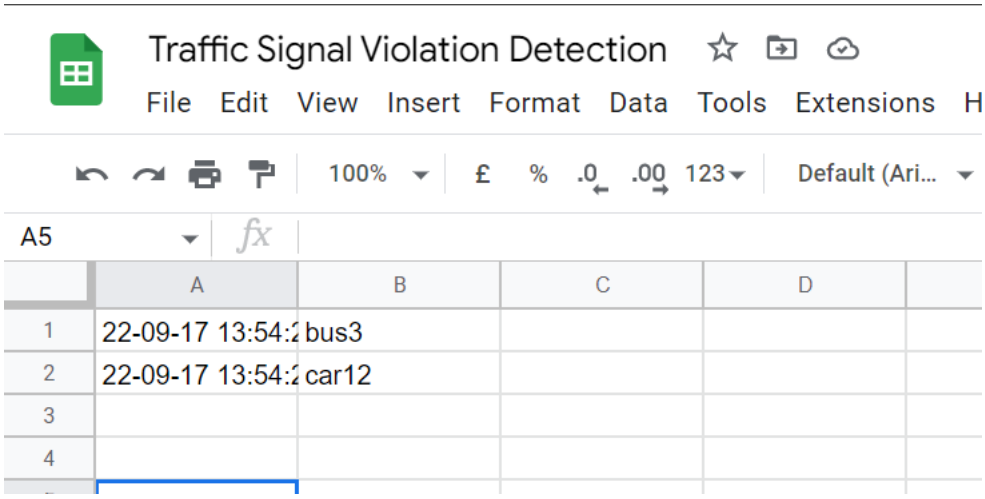


Figure 4.26: Excel sheet that storing the details of the vehicle

Here we can see the excel sheet. The A column stores the time and when the signal violation occurred and B column stored the tracking id and class name.

4.3 Summary

In this chapter the results and screen shots that are obtained by the system is described briefly.

Firstly the Results of vehicle detection then vehicle tracking is described briefly. Then the results of the traffic offence detection system is described in three consecutive parts that satisfies the three objectives of the project.

CHAPTER 5

CONCLUSION AND FUTURE ASPECTS

This section specifies the advantages,limitations and future scope of this project.

5.1 Conclusion

A traffic safety system based on computer vision has been introduced in this research. Real-time monitoring was done to keep track of moving objects and, if necessary, identify traffic infractions (such as excessive speeding, signal infractions, unauthorised parking). As a starting point for a road safety system, we examined the vehicle detection, categorization, and tracking approaches.

Speed is estimated when a vehicle stays at least five frames in the video. The change of vehicle's position in the x-axis and y-axis is considered for five consecutive frames, and speed is estimated in terms of pixel per second. When the total area covered by the traffic camera is available to the system, it can also calculate the estimated speed in terms of meter per second.

The traffic signal is synchronized with our system, and it monitors the road intersection points for a possible signal violation when the traffic light indicates red. A visual line on the intersection point of the road works as a reference for our system, and when any vehicle crosses the line while the traffic light indicates red, it is marked and logged for traffic signal violation. The vehicle remains marked even if it reverses back after violating the traffic signal.

: Vehicles that stand still on the main road and cause roadblocks are marked and logged for unauthorized parking. All the vehicle's IDs and positions from the previous frames are stored in the system. The current position of each detected vehicle is compared with the positions they were on previous frames to detect how long the vehicles stood still in the same places. Depending on the time the vehicle is standing idle on the street, the

vehicle is marked for unauthorized parking.
Live video may be recorded using existing CCTV cameras and then transmitted onto a central server for further analysis. The suggested model may be merged with a system that has better hardware and a greater accuracy level in further work. It may be useful for automated accident monitoring and traffic planning

5.2 Future Scopes

1. It is possible to combine the suggested model with a system that has superior hardware and a greater degree of precision.

2. Further functionality can be added to the system like-

    - Lane Violation
    - Unauthorized Vehicles Detection
    - Wrong Way Driving
    - Motor Bike Rider Without Helmet

REFERENCES

[1] Bochkovskiy, Alexey and Wang, Chien-Yao and Liao, Hong-Yuan Mark(2020), “YOLOv4: Optimal Speed and Accuracy of Object Detection “.publisher = arXiv year = 2020, Available:https://arxiv.org/abs/2004.10934,

[2] Global status report on road safety 2018. Geneva: World Health Organization; 2018. Licence: CC BY-NC-SA 3.0 IGO, Available:https://www.who.int/publications/i/item/9789241565684,

[3] G. Ou, Y. Gao and Y. Liu, "Real-Time Vehicular Traffic Violation Detection in Traffic Monitoring Stream," 2012 IEEE/WIC/ACM International Conferences on Web Intelligence and Intelligent Agent Technology, 2012, Macau, China, December 4-7, 2012. pp. 15-19, doi: 10.1109/WI-IAT.2012.91. Available:https://ieeexplore.ieee.org/document/6511640

[4] D. Beymer, P. McLauchlan, B. Coifman and J. Malik, "A real-time computer vision system for measuring traffic parameters," Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 1997, San Juan, PR, USA, June 17-19, 1997. pp. 495-501, doi: 10.1109/CVPR.1997.609371. Available:https://ieeexplore.ieee.org/document/609371

[5] G. Xiaoling Wang, Li-Min Meng, Biaobiao Zhang, Junjie Lu and K. -. Du, "A video-based traffic violation detection system," Proceedings 2013 International Conference on Mechatronic Sciences, Electric Engineering and Computer (MEC), 2013, Shenyang, China, December 20-22, 2013. pp. 1191-1194, doi: 10.1109/MEC.2013.6885246. Available:https://ieeexplore.ieee.org/document/6885246

[6] H. Zhu, C. Xu and F. Li, "The Traffic Volume Count Algorithm Based on Computer Vision," 2013 5th International Conference on Intelligent Human-Machine Systems and Cybernetics, 2013, Hangzhou, China, August 26-27, 2013. pp. 130-133, doi: 10.1109/IHMSC.2013.38. Available:https://ieeexplore.ieee.org/document/6643850

[7] L. Y. Deng, N. C. Tang, Dong-liang Lee, Chin Thin Wang and Ming Chih Lu, "Vision based adaptive traffic signal control system development," 19th International Conference on Advanced Information Networking and Applications (AINA'05) Volume 1 (AINA papers), 2005, Taipei, Taiwan, March 28-30, 2005. pp. 385-388 vol.2, doi: 10.1109/AINA.2005.342. Available:https://ieeexplore.ieee.org/document/1423717

[8] X. Wen-juan and L. Jian-feng, "Application of vision sensing technology in urban intelligent traffic control system," 2018 4th International Conference on Computer and Technology Applications (ICCTA), 2018, Istanbul, Turkey, May 3-5, 2018. pp. 74-77, doi: 10.1109/CATA.2018.8398659. Available:https://ieeexplore.ieee.org/document/8398659

[9] J. -a. Kim, J. -Y. Sung and S. -h. Park, "Comparison of Faster-RCNN, YOLO, and SSD for Real-Time Vehicle Type Recognition," 2020 IEEE International Conference on Consumer Electronics - Asia (ICCEAsia), 2020, Seoul, South Korea, November 1-3, 2020. pp. 1-4, doi: 10.1109/ICCE-Asia49877.2020.9277040. Available:https://ieeexplore.ieee.org/document/9277040

[10] V. Zinchenko, G. Kondratenko, I. Sidenko and Y. Kondratenko, "Computer Vision in Control and Optimization of Road Traffic," 2020 IEEE Third International Conference on Data Stream Mining Processing (DSMP), 2020, Lviv, Ukraine, August 21-25, 2020. pp. 249-254, doi: 10.1109/DSMP47368.2020.9204329. Available:https://ieeexplore.ieee.org/document/9204329

[11] N. Wojke, A. Bewley and D. Paulus, "Simple online and realtime tracking with a deep association metric," 2017 IEEE International Conference on Image Processing (ICIP), 2017, Beijing, China, September 17-20, 2017. pp. 3645-3649, doi: 10.1109/ICIP.2017.8296962. Available:https://ieeexplore.ieee.org/document/8296962

[12] OpenCV. 2020. opencv (Version 4.5.0) [Source code] Available:https://github.com/opencv/opencv

[13] The AI Guy. 2020. yolov4-deepsort [Source code]. Available:https://github.com/theAIGuysCode/yolov4-deepsort

[14] Simple Online and Realtime Tracking with a Deep Association Metric]. Available:https://arxiv.org/abs/1703.07402

[15] MaryamBoneh Vehicle-Detection Public. Available:https://github.com/MaryamBoneh/Vehicle-Detection

[16] YOLOv4 model architecture Available:https://iq.opengenus.org/yolov4-model-architecture/

[17] A Practical Guide to Object Detection using the Popular YOLO Framework – Part III (with Python codes),Pulkit Sharma — Published On December 6, 2018 and Last Modified On August 26th, 2021 Available:https://www.analyticsvidhya.com/blog/2018/12/practical-guide-object-detection-yolo-framewor-python/

BTP Report

by Maneesh Mondal

Submission date: 20-Sep-2022 03:33PM (UTC+0530)
Submission ID: 1904410704
File name: BTP_MONDAL.pdf (8.85M)
Word count: 9358
Character count: 48251

BTP Report

ORIGINALITY REPORT

10%	8%	7%	%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Fahimul Hoque Shubho, Fahim Iftekhar, Ekhfa Hossain, Shahnewaz Siddique. "Real-time traffic monitoring and traffic offense detection using YOLOv4 and OpenCV DNN", TENCON 2021 - 2021 IEEE Region 10 Conference (TENCON), 2021 Publication	2%
2	www.researchgate.net Internet Source	2%
3	www.analyticssteps.com Internet Source	1%
4	dissertations.port.ac.uk Internet Source	1%
5	www.slideshare.net Internet Source	1%
6	B. A. Sujatha Kumari, A. G. Varun Yadiyala, B. J. Aruna, C. Radha, B. Shwetha. "Chapter 36 Early Detection of Mild Cognitive Impairment Using 3D Wavelet Transform", Springer Science and Business Media LLC, 2021 Publication	1%

7	www.coursehero.com Internet Source	<1%
8	www.editcode.net Internet Source	<1%
9	repository.tudelft.nl Internet Source	<1%
10	J. Malik. "A real-time computer vision system for measuring traffic parameters", Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition CVPR-97, 1997 Publication	<1%
11	nms.kcl.ac.uk Internet Source	<1%
12	repository.iiitd.edu.in Internet Source	<1%
13	"S290 topic 3 rare diseases WEB077020", Open University Publication	<1%
14	www.astesj.com Internet Source	<1%
15	researchrepository.wvu.edu Internet Source	<1%
16	Bram Alefs, David Schreiber. "Accurate Speed Measurement from Vehicle Trajectories using	<1%

	AdaBoost Detection and Robust Template Tracking", 2007 IEEE Intelligent Transportation Systems Conference, 2007 Publication	
17	catalog.uttyler.edu Internet Source	<1 %
18	www.igi-global.com Internet Source	<1 %
19	www.scribd.com Internet Source	<1 %
20	Habib Izadkhah. "Basic structure of neural networks", Elsevier BV, 2022 Publication	<1 %
21	Zhu, Huasheng, Chenguang Xu, and Fan Li. "The Traffic Volume Count Algorithm Based on Computer Vision", 2013 5th International Conference on Intelligent Human-Machine Systems and Cybernetics, 2013. Publication	<1 %
22	cag.gov.in Internet Source	<1 %

Exclude quotes	On	Exclude matches	< 10 words
Exclude bibliography	On		